SSPL ERP

Backup, Restore, Update & Rollback — Installation and Usage Guide

ERPNext v16 | Ubuntu Server 24.04 + Docker Compose | July 2026

Contents

1. Overview & server layout
2. Prerequisites
3. Fresh server — start here
4. Part 1 — Web admin panel
5. Part 2 — Backup system: installation
6. Part 2 — Backup system: daily use
7. Part 2 — Restoring from a backup
8. Part 2 — Optional cloud upload (rclone)
9. Part 3 — Update & rollback system: installation
10. Part 3 — Updating the system
11. Part 3 — Rolling back an update
12. Part 3 — Managing image backups
13. Scheduled jobs summary
14. Keeping this toolkit up to date
15. Quick command reference
16. Troubleshooting

1. Overview & server layout

This toolkit has three independent parts:

- **Backup system** (`Backup/frappe_backup_system/`) — automated daily backups of the ERPNext **database, uploaded files (public and private), and configuration**, plus a restore script. Installed to `/opt/scripts/v2/`, backups stored in `/opt/backups/frappe/`.
- **Update & rollback system** (`Production Installation/update and rollback/`) — safely pulls new Docker images from GHCR, taking a full data backup and a Docker image snapshot first, so a bad update can be rolled back. Installed to `/opt/sspl-erp/v2/`.
- **Web admin panel** (`Admin Panel/`) — a browser interface (HTTPS, admin login) that can **install ERPNext and the other two parts from the browser** (Setup switches), then runs the backup / update / rollback scripts, shows server health and backup files, clears RAM caches, and accepts backup file uploads. Installed to `/opt/sspl-admin/`, served on port **8090**. See section 4.

Setting up a **brand-new server**? Install the **admin panel** first, then install ERPNext and the other two parts from the browser — see section 3. (A one-command terminal installer that does everything at once is available too.) Everything can be refreshed later in one command with `./update_tooling.sh` — see section 14.

Both parts install into their own `v2/` folder, **side-by-side** with any older scripts already on the server: nothing in `/opt/scripts/` or directly in `/opt/sspl-erp/` is replaced, so the old scripts remain available as a fallback. Backup *data* is shared — both generations store data backups in `/opt/backups/frappe/` and image snapshots in `/opt/sspl-erp/image-backups/`, so a backup made by one can be restored with the other.

```
Your server
├── /opt/sspl-erp/                    ← Docker deployment (docker-compose.yml, .env)
│   ├── sspl-erp-*.sh                 ← old update scripts (kept as fallback)
│   ├── v2/                           ← new update & rollback scripts
│   │   ├── sspl-erp-common.sh        ← shared config: SITE_NAME lives here
│   │   ├── sspl-erp-update-with-rollback.sh
│   │   ├── sspl-erp-rollback.sh
│   │   └── sspl-erp-backup-manager.sh
│   └── image-backups/                ← Docker image snapshots (shared, rollback)
├── /opt/sspl-admin/                  ← web admin panel (app, config.json, certs/, jobs/)
```

```
├── /opt/scripts/                      ← old backup scripts (kept as fallback)
│   └── v2/                            ← new backup scripts
│       ├── frappe_backup.sh           ← full backup (DB + files + config)
│       ├── frappe_db_backup.sh        ← quick database-only backup
│       ├── frappe_backup_verify.sh    ← checks backups are complete & fresh
│       ├── frappe_restore.sh          ← restore a full backup
│       └── frappe_migrate.sh          ← run bench migrate on its own
└── /opt/backups/frappe/               ← data backups (shared)
    ├── 20260712_020000/               ← one folder per full backup
    │   ├── *-database.sql.gz          ← database dump
    │   ├── *-files.tgz                ← public files
    │   ├── *-private-files.tgz        ← private files / attachments
    │   ├── site_config.json, .env, docker-compose.yml
    │   └── backup_manifest.txt
    └── db-only/                       ← 6-hourly database dumps
```

2. Prerequisites

- Ubuntu Server 24.04 with Docker Engine + Docker Compose v2 (`docker compose` , not `docker-compose`).
- The ERPNext stack deployed at `/opt/sspl-erp/` with a `docker-compose.yml` and a `.env` file containing `MARIADB_ROOT_PASSWORD` .
- Root / sudo access on the server.
- Your Frappe **site name**. To find it:

```
docker compose -f /opt/sspl-erp/docker-compose.yml exec backend ls sites/
```

The site name is the folder that is not `apps.txt` , `assets` , or `common_site_config.json` — for example `192.168.225.135` .

3. Fresh server — start here

On a **new server** (nothing deployed yet) the only thing you install from the command line is the **web admin panel**. Everything else — ERPNext itself, the backup system, the update & rollback scripts — is then installed from the panel, in the browser. Requirements: Ubuntu Server with Docker installed and a static LAN IP.

Step 1 — Install the admin panel (terminal, once)

```
git clone https://github.com/santhosh3279/sspl_installation.git
cd sspl_installation/"Admin Panel"
chmod +x setup_admin_panel.sh
./setup_admin_panel.sh
```

It asks for an admin username, password and port (default **8090**), generates the HTTPS certificate, records where the repo is checked out, and starts the `sspl-admin` service. Takes about a minute. Full details in section 4.

Step 2 — Install ERPNext from the panel

Open `https://<server-ip>:8090` , click through the self-signed certificate warning (*Advanced → Proceed*) and log in. The **Setup — install components** card at the top of the page drives the whole rest of the installation, in order:

1. **Install ERPNext** — enter the server IP, HTTP port (default **80**), MariaDB root password and ERPNext Administrator password, then click the button. The panel clones `frappe_docker` , writes `.env` and the pdf-fix override, generates `/opt/sspl-erp/docker-compose.yml` , pulls the `sspl-erpnext` image from GHCR, starts the containers and waits until the database and backend are really ready, then creates the site named after the server IP with **every app shipped in the image** installed automatically (erpnext, hrms, india_compliance, ssplbilling, printer_server_configuration, frappe_whatsapp, …), sets it as the default site and repairs the MariaDB grants. This takes 10–20 minutes and streams live into the terminal at the foot of the page.
2. **Install backup system** — one click: Part 2 in `/opt/scripts/v2/` with its cron jobs.
3. **Install update & rollback scripts** — one click: Part 3 in `/opt/sspl-erp/v2/` with the site name pre-filled.

Steps 2 and 3 unlock once ERPNext is installed and reuse the deployed site name automatically; each switch turns into an *installed* ✓ status when it finishes. That is the entire installation — the same page is then your day-to-day console (section 4), and the manual installation steps in sections 5 and 9 are only for installing Parts 2 and 3 by hand instead.

Won't double-schedule backups. Before installing the v2 backup cron jobs — from the panel or the terminal — the installer checks the crontab for an existing entry pointing at `/opt/scripts/` outside `/opt/scripts/v2/`, e.g. an older backup script wired up by hand (the “Santhosh additions” in this guide's history). If one is found, the v2 scripts are still installed but their cron jobs are **not** scheduled automatically, and the conflicting line is printed so you can switch over manually with `sudo crontab -e` (see section 13 for the v2 schedule).

Alternative — all-in-one terminal installer

If you would rather do the whole thing from the command line instead, one script performs the entire production deployment — the whole *SSPL ERP Production Deployment Guide* — plus Parts 1–3 of this toolkit, in a single unattended run:

```
cd "sspl_installation/Production Installation"
chmod +x install_sspl_erp.sh
./install_sspl_erp.sh
```

All questions are asked **up front** — server IP, HTTP port, MariaDB root password, Administrator password, which components to install, and the admin panel login. After the final *Proceed?* it runs for 10–20 minutes and installs exactly the same things as the panel-first flow above, plus Docker auto-start on reboot, docker group membership, and passwordless sudo shortcuts for clear-RAM and manual backup. Use this **or** the panel-first flow — not both.

Already installed? It refuses to run. If ERPNext is already deployed and running on the server, this installer prints *“already installed”* and exits without changing anything — use the update tools instead (sections 10 and 14). To deliberately re-run, e.g. to finish a half-completed install, use `SSPL_FORCE=1 ./install_sspl_erp.sh`; a forced re-run is safe — the existing site is never re-created, and existing `.env` files are backed up before being rewritten.

4. Part 1 — Web admin panel

A browser interface for everything above: **install the whole system from the browser**, then run backups, updates, and rollbacks with one click, watch the script output live, browse and download backup files, monitor CPU / memory / disk / container health, clear RAM caches, and upload backup files to the server.

Installation

```
git clone https://github.com/santhosh3279/sspl_installation.git
cd sspl_installation/"Admin Panel"
chmod +x setup_admin_panel.sh
./setup_admin_panel.sh
```

The installer asks for an admin username, password, port (default **8090**), and the server IP for the HTTPS certificate. It installs a Python environment to `/opt/sspl-admin/`, records the repo location (`repo_dir` in its config), and starts a systemd service (`sspl-admin`) that survives reboots — see *Autostart* below.

> **The panel can install everything else** — Parts 2 and 3 do *not* need to be in place first. Keep the git checkout on the server (the Setup switches run the installers from it; `git pull` there keeps them current).

Panel-first setup

Open `https://<server-ip>:8090` and log in. The certificate is self-signed (a LAN IP cannot get a public certificate), so the browser warns the **first** time — click *Advanced → Proceed*; the connection is fully encrypted either way. To remove the warning on a particular PC, import `/opt/sspl-admin/certs/sspl-admin.crt` into its trusted store.

The page is one column: the **control panel** (setup, server health, actions, backups) with a full-width **terminal** at the foot of the page showing the live output of whatever is running. Starting a job scrolls the terminal into view; drag its bottom edge to make it taller.

Every run is also written to a log file on the server, so nothing is lost when the page is closed or the browser is refreshed. *Download Log* beside the terminal heading saves the current run's log; **Log history — past runs** just below the terminal folds open into every earlier run, newest first, each with its own download link. Send that file when reporting a failed install or backup. The logs can quote configuration and backup contents, so treat them as confidential.

The **Setup — install components** card at the top shows what is installed and lets you install the rest, in order:

1. **ERPNext stack** — enter the server IP, HTTP port, MariaDB root password, and Administrator password, then click *Install ERPNext*. This pulls the image, starts the containers, and creates the site (10–20 minutes); progress streams into the terminal below. The passwords are sent over HTTPS as environment variables and are never written to the log.
2. **Backup system** and **Update & rollback scripts** — enabled once ERPNext is installed, and reuse the deployed site name automatically. One click each.

Each switch becomes an *installed* ✓ status when done.

Day-to-day use

- **Actions** — full backup, DB-only backup, verify, system update, rollback (with a snapshot picker), **Run migrations**, and a Clear-RAM-caches button. Only one job runs at a time; its output streams into the terminal at the foot of the page. Run migrations is the retry for an update or restore that failed at the migration step and nothing else — it is the same `bench migrate` those two run internally.
- **Backups** — full backups with DB / Files / Private completeness badges, DB-only dumps, image snapshots, and uploads; every file can be downloaded. A full backup also has a **Download** button beside its Restore button that saves the whole folder — database dump and file archives together — as one `.zip`, which is what you want when moving a backup to another machine.
- **Upload** — send backup files to the server; they are stored under `/opt/backups/frappe/uploads/` and never executed or restored automatically. Each uploaded file and folder has a **Delete** button to remove it again. Deleting only ever works inside `uploads/` — real backups and image snapshots have no delete button and cannot be removed from the browser.

Restoring from the panel

Restore is the one action that destroys data, so it is not a plain button. Full backups and uploaded backup *folders* that contain a `*-database.sql.gz` get a **Restore** button; clicking it opens a warning naming the exact site about to be

overwritten, and asks you to type the **site name** in full and re-enter your **admin panel password**. Both are checked on the server. The job then takes a **full safety backup first** — if that backup fails, the restore never starts — and only then begins the restore, which asks for the **MariaDB root password** in the terminal. Type it into the input line under the terminal and press Enter, then answer `yes` to the final confirmation. What you type never reaches the job log, but the input line is an ordinary text box, so the password *is* visible on your own screen as you type it — unlike a console, which hides it.

> **To abort a restore** that is waiting at a prompt, answer `no` at the confirmation. If nobody is there to answer, `sudo systemctl restart sspl-admin` ends it. While it waits, the one-job-at-a-time rule blocks the panel's other buttons; scheduled cron backups are unaffected.

> To restore an **uploaded** backup, upload it into its own named folder (the *Folder* box on the upload form) so its database and files stay together — a whole folder is what gets restored. The console route still works and is unchanged (section 7).

Managing the service

```
sudo systemctl status sspl-admin       # is it running?
sudo systemctl restart sspl-admin      # restart
sudo journalctl -u sspl-admin -f       # follow the logs
```

To change the password or port, re-run `./setup_admin_panel.sh` — it keeps the certificate and Python environment. Keep port 8090 LAN-only; never port-forward it to the internet.

Autostart — the panel runs on its own

The panel is **not** a program you keep open in a terminal. The installer finishes with `systemctl enable --now sspl-admin`, so the service:

- starts **automatically at boot**, before anyone logs in;
- keeps running after you close the terminal window, log out, or drop the SSH session;
- restarts itself if it crashes (`Restart=on-failure`, 5-second delay).

Confirm autostart is armed — `enabled` means it comes back after a reboot, `active` means it is running right now:

```
systemctl is-enabled sspl-admin        # -> enabled
systemctl is-active sspl-admin         # -> active
```

If it ever reports `disabled`, turn it back on with `sudo systemctl enable --now sspl-admin`. To stop the panel starting at boot (for maintenance, say), use `sudo systemctl disable --now sspl-admin` — that stops it now *and* keeps it stopped after a reboot.

> **Do not run `python app.py` by hand once the service is installed.** Two copies would fight over port 8090, and the hand-started one dies with the terminal. The service runs the *installed* copy at `/opt/sspl-admin/app.py`, not the file in the git checkout — so after editing the panel in the repo, either re-run `./setup_admin_panel.sh` or copy it across and restart.

```
sudo cp app.py /opt/sspl-admin/     # after editing the repo copy
sudo systemctl restart sspl-admin
```

5. Part 2 — Backup system: installation

Step 1 — Copy the files to the server

```
# From the repo, copy either the folder or the tarball to the server, then:
tar -xzf frappe_backup_system.tar.gz
cd frappe_backup_system           # or: cd Backup/frappe_backup_system
```

Step 2 — Run the installer

```
chmod +x setup_frappe_backups.sh
sudo ./setup_frappe_backups.sh
```

The installer will:

1. Create `/opt/backups/frappe/` (permissions restricted to root — backups contain credentials).
2. Install the four scripts to `/opt/scripts/v2/` and make them executable — older scripts directly in `/opt/scripts/` are not touched.
3. **Ask for your site name** and write it into the backup, db-backup, and restore scripts.
4. Offer to install the cron jobs (see section 13 for the schedule).
5. Offer to run a test backup (answer `yes` the first time).

If the old backup scripts are already scheduled in cron: installing the new cron jobs too means every backup runs twice. Either answer `no` to the cron question while you test the v2 scripts manually, or say `yes` and then remove the old `/opt/scripts/frappe_*.sh` lines with `sudo crontab -e` once you are happy with v2.

Step 3 — Verify the test backup

```
sudo /opt/scripts/v2/frappe_backup_verify.sh
```

All four lines must say **OK**: database, public files, private files, and config. The script exits with an error if anything is missing.

6. Part 2 — Backup system: daily use

Once installed, backups run automatically from cron. Manual commands:

Task	Command
Full backup now (DB + files + config)	`sudo /opt/scripts/v2/frappe_backup.sh`
Quick database-only backup	`sudo /opt/scripts/v2/frappe_db_backup.sh`
Check backups are complete & recent	`sudo /opt/scripts/v2/frappe_backup_verify.sh`
See what is stored	`ls -lh /opt/backups/frappe/`
Watch the backup log	`tail -f /var/log/frappe_backup_v2.log`

Retention is automatic: full backups are kept **30 days**, database-only dumps **14 days**. The newest **20** of each survive regardless of age, so backups that stop running for longer than the retention window can never leave you with an empty backup folder — the same figure the panel's *Clear backups older than 30 days* button keeps. If a cloud remote is configured, it keeps the newest **10** of each as well — counted, not dated, so cloud storage stays bounded. To change this, edit `RETENTION_DAYS`, `LOCAL_KEEP_MIN` or `CLOUD_KEEP` at the top of the respective script in `/opt/scripts/v2/`.

Good habit: run a manual full backup before any risky change (new app install, configuration change, server maintenance) — and always before an update, although the update script also does this automatically.

7. Part 2 — Restoring from a backup

> **Warning:** a restore **overwrites** the site's current database and files with the backup contents. Anything entered after the backup was taken is lost.

You can restore either **from the admin panel** — which takes a safety backup first and is described in section 4 — or from the console as below. The console route is what you want when the panel itself is unavailable.

Step 1 — Pick a backup

```
ls /opt/backups/frappe/
# e.g. 20260712_020000
```

Step 2 — Get the MariaDB root password

```
grep MARIADB_ROOT_PASSWORD /opt/sspl-erp/.env
```

Step 3 — Run the restore

```
sudo /opt/scripts/v2/frappe_restore.sh /opt/backups/frappe/20260712_020000
```

The script will:

1. Ask for the site name (press Enter to accept the configured default).
2. Ask for the MariaDB root password (input is hidden).
3. Ask for a final `yes` confirmation.
4. Restore the database, public files, and private files in one operation.
5. Run `bench migrate` (in case app versions differ from the backup) and restart all services.

Afterwards, log in to ERPNext and verify recent documents, attachments, and print formats.

If the restore stops at the migration

A restore is two steps: loading the data, then `bench migrate` so the patches match the app code in the running image. They fail differently, and the script says which one you got.

> **"The data was restored, but the migration did NOT finish."** The data loaded correctly; what did not finish is the patching, so the database is part-patched. What failed is a patch belonging to one of the apps — usually because the code in the image is older or newer than the data. **Do not restore again**: that only puts you back at this same point, since the database was never the problem.

Read the traceback for the app and patch it names, fix or update that app, then re-run only the migration — from the panel's **Run migrations** button, or:

```
sudo /opt/scripts/v2/frappe_migrate.sh
```

If the patch is known-broken upstream and you accept a part-patched database until it is fixed:

```
sudo SSPL_MIGRATE_SKIP_FAILING=yes /opt/scripts/v2/frappe_migrate.sh
```

Or, if this bench has the command (check with `bench --site <your-site> --help`), mark that one patch as done, permanently, and migrate again:

```
docker compose -f /opt/sspl-erp/docker-compose.yml exec backend \
    bench --site <your-site> bypass-patch <patch.module.name>
```

> **"The restore did NOT finish."** No such reassurance — the old database is gone and the new one is partial. Restore again, from the safety backup the panel took immediately before this attempt.

8. Part 2 — Optional cloud upload (rclone)

By default backups stay on the server's own disk. To survive a disk failure, upload them to cloud storage. Full provider walkthrough: see *Rclone_Configuration_Guide.docx*.

Step 1 — Install and configure rclone (one time)

```
sudo apt install rclone
rclone config          # create a remote, e.g. named "gdrive"
```

Step 2 — Point the backup script at the remote

Edit `/opt/scripts/v2/frappe_backup.sh` and set the variable near the top:

```
RCLONE_REMOTE="gdrive:frappe-backups"
```

Every full backup is then uploaded automatically after it completes. If the upload fails, the backup still exists locally and a warning is written to the log.

Step 3 — Test it

```
sudo /opt/scripts/v2/frappe_backup.sh
rclone ls gdrive:frappe-backups | head
```

9. Part 3 — Update & rollback system: installation

Step 1 — Copy the four scripts

Copy these from `Production Installation/update and rollback/` into `/opt/sspl-erp/v2/` (create it with `sudo mkdir -p /opt/sspl-erp/v2`). Older `sspl-erp-*.sh` scripts directly in `/opt/sspl-erp/` stay untouched as a fallback; both versions share the same `image-backups/` snapshots.

- `sspl-erp-common.sh` — shared configuration (sourced by the others, not run directly)
- `sspl-erp-update-with-rollback.sh`
- `sspl-erp-rollback.sh`
- `sspl-erp-backup-manager.sh`

Step 2 — Set the site name

Edit `/opt/sspl-erp/v2/sspl-erp-common.sh` and set the variable at the top (this is the **only** place it needs changing):

```
SITE_NAME="192.168.225.135"
```

Step 3 — Make the scripts executable

```
cd /opt/sspl-erp/v2
chmod +x sspl-erp-update-with-rollback.sh sspl-erp-rollback.sh sspl-erp-backup-manager.sh
```

> **Note:** the update script calls `/opt/scripts/v2/frappe_backup.sh`, so install Part 2 first.

10. Part 3 — Updating the system

When a new image has been published to GHCR (`ghcr.io/santhosh3279/sspl-erpnext`):

```
cd /opt/sspl-erp/v2
./sspl-erp-update-with-rollback.sh
```

What it does, in order:

1. Runs a **full Frappe data backup** (database + files). If this fails you are asked whether to continue.
2. Snapshots the current Docker images to `/opt/sspl-erp/image-backups/backup_TIMESTAMP.tar`.
3. Prunes unused Docker resources and **pulls the latest images with the site still running** — the download is the longest step of an update, so it is kept out of the downtime window. If the pull fails nothing has been changed and the site stays up on the current images.
4. **Checks that no installed app is missing from the new image** — the migration would fail on it. If one is, the update stops before anything changes; uninstall the app first (see below).
5. Stops all services, then starts them on the new images and **waits until the database and backend are actually ready** (up to 3 minutes).
6. Repairs MariaDB user grants.
7. Takes the user-facing services (frontend, websocket, queues, scheduler) **offline** and runs `bench migrate`. Step 4 started everything, so without this window users would be on a site whose code is new and whose schema is still moving. The site is brought back up whatever happens — a failure, a Ctrl-C or a kill cannot leave it dark.
8. **Installs any app the new image has added** that the site does not already have — `bench migrate` only touches apps the site already runs, so without this an app added to the image would never appear. Apps already installed are left alone.
9. Clears the cache.
10. Prints the new version and deletes image snapshots older than the 3 most recent.

If any step fails, the script stops immediately and prints rollback instructions.

> **If the update fails at the migration:** the migration is the one step that changes the database, and a patch belonging to any app in the new image can fail there. The script stops, puts the site back online, and deliberately does **not** install new apps or clear caches on top of a part-migrated database. You are then running the new images against a database

whose patches did not finish, so expect errors until you do one of:

1. **Fix the app that failed** (the traceback names it), then re-run only the migration, leaving the new images in place: `sudo /opt/scripts/v2/frappe_migrate.sh`
2. **Roll the images back**: `/opt/sspl-erp/v2/sspl-erp-rollback.sh`

Rollback is second, not first, because it is not a clean undo here: `bench migrate` synchronises the schema *before* it runs the post-sync patches, so by the time a patch fails the tables have already been altered — and the rollback restores images only (section 11). Old code against a moved schema usually works, because Frappe tolerates columns it does not know about, but it is not the matching pair it looks like. If it cannot cope, restore the data backup the update took at the start. Restoring is not the first thing to reach for either: the data is intact, the patches are what is missing. See section 7 for the `--skip-failing` and `bypass-patch` escape hatches.

Note: installing a new app writes to the database, and a rollback only restores the Docker images (section 11) — it will not remove an app that installed. If an update fails while installing one, restore the data backup the update took at the start (section 7). When an image adds an app for the first time, rehearse the update on a test server restored from a production backup.

Removing an app from the image: uninstall it from the site **first**, while the running image still has its code. Uninstalling **deletes the app's DocTypes and all their data**. To remove one (for example `frappe_whatsapp`): export any of its data you want to keep, pick it next to **Uninstall app** in the Admin Panel and press the button (or run `sudo /opt/sspl-erp/v2/sspl-erp-uninstall-app.sh frappe_whatsapp`). It takes a full backup first and stops if that fails. Then publish the image without the app and run the update. Until the image drops the app, updates do not reinstall it. If the uninstall fails, the database may be half changed: fix the cause and run it again, or restore the backup it took (section 7).

Verify after every update

```
docker compose ps
docker compose logs --tail=50
docker compose exec backend bench version
docker compose exec backend bench --site 192.168.225.135 doctor
```

Then log in and check the key business flows (e.g. create a draft invoice, open a report, test printing).

11. Part 3 — Rolling back an update

If the system misbehaves after an update:

```
cd /opt/sspl-erp/v2
./sspl-erp-rollback.sh
```

The script shows the most recent image snapshot (size and date), asks for confirmation, stops the services, loads the saved images, restarts, repairs DB grants, and clears the cache.

Rolling back to a specific older snapshot

```
./sspl-erp-backup-manager.sh list
BACKUP_FILE=/opt/sspl-erp/image-backups/backup_20260701_090000.tar ./sspl-erp-rollback.sh
```

Important: rollback restores the Docker **images** only — not the database. If the failed update's migration changed the database structure, also restore the data backup that the update script took automatically (section 7). Its timestamp is printed at the start of the update.

12. Part 3 — Managing image backups

Image snapshots are several GB each. The update script already keeps only the last 3; the manager gives manual control:

Task	Command
List snapshots (marks the latest)	`./sspl-erp-backup-manager.sh list`
Keep newest 5, delete the rest	`./sspl-erp-backup-manager.sh clean 5`
Same, without confirmation (for cron)	`./sspl-erp-backup-manager.sh clean 5 --yes`
Delete all snapshots	`./sspl-erp-backup-manager.sh delete-all`
Check disk space	`df -h /opt/sspl-erp`

Note: when run from cron there is no terminal to confirm on, so `--yes` is required — without it the command exits with an error instead of deleting.

13. Scheduled jobs summary

Installed by the Part 2 setup script (view with `sudo crontab -l`):

Schedule	Job	Log file
Daily, 02:00	Full backup — `frappe_backup.sh`	`/var/log/frappe_backup_v2.log`
Every 6 hours	DB-only backup — `frappe_db_backup.sh`	`/var/log/frappe_db_backup_v2.log`
Sunday, 03:00	Backup verification — `frappe_backup_verify.sh`	`/var/log/frappe_backup_verify_v2.log`

Optional — weekly cleanup of image snapshots (add with `sudo crontab -e`):

```
0 2 * * 0 /opt/sspl-erp/v2/sspl-erp-backup-manager.sh clean 5 --yes >> /var/log/sspl-erp-backup-clean.log 2>&1
```

14. Keeping this toolkit up to date

When this repository gets improvements, refresh everything installed on the server in one command:

```
cd sspl_installation
git pull
./update_tooling.sh
```

It re-installs the backup scripts (`/opt/scripts/v2/`), the update/rollback scripts (`/opt/sspl-erp/v2/`), and the admin panel (`/opt/sspl-admin/` , restarting and health-checking its service). It is fully non-interactive and **preserves all of your configuration**: site name, rclone remote, retention days, alert email, the panel's login, port, and certificate. Cron jobs are not changed, and components that were never installed are skipped with a pointer to the right installer.

15. Quick command reference

I want to…	Run
Set up a brand-new server	`cd "Admin Panel" && ./setup_admin_panel.sh` — then install ERPNext from the panel
Set up a brand-new server, all from the terminal	`cd "Production Installation" && ./install_sspl_erp.sh`
Back up everything now	`sudo /opt/scripts/v2/frappe_backup.sh`
Back up only the database	`sudo /opt/scripts/v2/frappe_db_backup.sh`
Check my backups are OK	`sudo /opt/scripts/v2/frappe_backup_verify.sh`
Restore a backup	`sudo /opt/scripts/v2/frappe_restore.sh /opt/backups/frappe/<TIMESTAMP>`
Retry migrations after a failed update or restore	`sudo /opt/scripts/v2/frappe_migrate.sh`
Update to the latest release	`cd /opt/sspl-erp/v2 && ./sspl-erp-update-with-rollback.sh`
Undo the last update	`cd /opt/sspl-erp/v2 && ./sspl-erp-rollback.sh`
See image snapshots	`./sspl-erp-backup-manager.sh list`
Free disk space	`./sspl-erp-backup-manager.sh clean 3`
Open the admin panel	`https://<server-ip>:8090`
Restart the admin panel	`sudo systemctl restart sspl-admin`
Update all installed tooling	`git pull && ./update_tooling.sh`
Check services	`docker compose ps`
Check ERPNext version	`docker compose exec backend bench version`
View recent logs	`docker compose logs --tail=50`

16. Troubleshooting

Backup verification says something is MISSING

```
sudo tail -50 /var/log/frappe_backup_v2.log      # find the failing step
ls -lh /opt/backups/frappe/<latest>/          # see what was actually saved
sudo /opt/scripts/v2/frappe_backup.sh            # re-run manually and watch the output
```

Update failed partway through

The script prints instructions when it fails. In short: roll back the images with `./sspl-erp-rollback.sh`; if the migration had already modified the database, also restore the data backup taken at the start of the update (section 7).

Rollback cannot find or load a snapshot

```
ls -lh /opt/sspl-erp/image-backups/                    # is the .tar there?
tar -tvf /opt/sspl-erp/image-backups/backup_*.tar | head   # is it readable?
# Load manually if needed:
docker load -i /opt/sspl-erp/image-backups/backup_<TIMESTAMP>.tar
docker compose up -d
```

Site unreachable after update / rollback

```
docker compose ps                              # any container restarting or exited?
docker compose logs backend --tail=100
docker compose exec backend bench --site 192.168.225.135 doctor
```

Disk is filling up

```
du -sh /opt/sspl-erp/image-backups /opt/backups/frappe
./sspl-erp-backup-manager.sh clean 2           # image snapshots
# Data backup retention: lower RETENTION_DAYS in /opt/scripts/v2/frappe_backup.sh
```

Cron jobs not running

```
sudo crontab -l                                # jobs present?
grep CRON /var/log/syslog | tail               # did they fire?
sudo tail /var/log/frappe_backup_v2.log           # what did they say?
```